

Executive summary

A 4-member MCA team **can feasibly develop** a **mule-account detection solution within the hackathon timeline** by leveraging their strong programming and database skills. Key roles should include *data/ML engineer, backend developer, domain specialist*, and *devops/testing*. Problem 2 (suspicious transactions/ mules) is clearly defined; teams should set success as reliably flagging mule patterns in a simulated dataset and delivering a working prototype. Benefits include hands-on experience with streaming analytics and AI, plus networking; risks are steep learning curves (AML regulations, real-time systems) and data scarcity. An MVP might ingest synthetic transactions and alerts, compute features (e.g. transaction velocity, graph metrics), apply a basic ML model, and output risk scores. Required data are proprietary, so use public/synthetic data (e.g. SynthAML ¹, PaySim) or generate it. We recommend a simple microservices architecture: real-time stream ingestion (Kafka/Flink), feature store, ML inference (graph/Supervised model), and alert dashboard. Use Python ML (Scikit-learn, PyTorch), a graph DB (Neo4j), and Apache Kafka. Integrate with BOI's mock APIs; note hackathon IP rules (co-ownership with BOI/IITH ²) and no official BOI data sharing is specified (unspecified). Success hinges on clear deliverables: proposal, progress report, and a final demo showing realistic mule scenarios. Below is a detailed plan with tables, diagrams, and schedule.

Team feasibility and roles

An MCA team typically has strong programming/database knowledge but may lack banking domain or large-scale ML experience. Required skills:

- **Data engineering:** Stream processing, databases (NoSQL/SQL).
- **Machine Learning:** Building/validation of models (e.g. Python, Scikit-learn, PyTorch).
- **DevOps:** Setting up pipelines, Docker, Kafka clusters.
- **Domain/AML:** Understanding of banking transactions and AML concepts.

A possible team-role breakdown:

Role	Responsibility	Required Skills
Data/ML Engineer	Data pipeline, feature engineering, model training	Python, Pandas, Scikit-learn, PyTorch
Backend Developer	API integration, stream processing setup (Kafka/Flink)	Java/Python, Kafka/Flink, SQL/NoSQL
Domain Specialist	AML rules, KYC data insight, risk score logic	Finance/AML knowledge, compliance awareness
DevOps/Tester	Infrastructure, CI/CD, containerization, testing	Docker/Kubernetes, Git, Linux administration

This allocation assumes all can code, with one member focusing on learning AML basics. Given MCA coursework, students likely need to self-learn AML regulations and graph analytics. Short online courses or mentor input can help. The team must cover both software engineering and machine-learning expertise; if skills are unbalanced, consider an extra advisor.

Problem selection and criteria

The hackathon offers two problems; this report focuses on **Problem 2** (mule account detection) ³ . This is appropriate given its defined scope. Success can be measured by:

- **Detection accuracy:** Ability to flag mule accounts in test scenarios (e.g. $\geq 80\%$ of injected mule cases flagged).
- **False positive control:** Keeping benign accounts mostly unflagged (low FPR).
- **Real-time processing:** Near-real-time alert generation on simulated streaming data (<2 seconds latency).
- **Explainability:** Providing interpretable reasons (e.g. graph feature shift) for alerts.

An MVP can be: a stream processor that ingests transactions and outputs mule-alerts with justification. Full production-grade compliance is not expected; focus on prototyping the core logic.

Benefits and risks of participation

Benefits: Teams gain experience with streaming architectures and financial AI. Successful demo and prizes (~₹5 L for 1st place) ³ , plus networking (invitation to Global FinTech Fest). Prestigious collaboration (BOI, IITH) enhances résumés. It promotes practical skills in a high-impact domain.

Risks: The problem is complex; data scarcity can hinder realistic testing (no official BOI data given). AML domain complexity and unknown hackathon requirements (milestones, resources) add uncertainty. Time is tight (only ~3 months). Teams should mitigate by scoping MVP narrowly and leveraging synthetic data. Entry fee is free, so financial risk is nil; reputational risk is low but teams must manage expectations on outcomes.

Success criteria and MVP definition

Success Criteria: Clear end-goals enable focused development. We suggest:

- A working pipeline that **flags at least one simulated mule scenario** per hour of data streaming.
- Demonstrable reduction in false negatives (all known mule transfers in test set are caught).
- Generation of a concise risk report per alert (to aid explainability).

Minimum Viable Product (MVP):

- **Data Ingestion:** Simulate transaction stream (e.g. using Kafka), including normal and mule patterns.
- **Feature Computation:** Compute key features (transaction count, amount sums, recipient graph degree).
- **Model/Rules:** Implement either a simple ML classifier or a rule (e.g. “account with >N transfers to new accounts in short period”) to score mule-likelihood.
- **Output:** Risk scores/alerts with rationale. E.g. “Account A flagged: sudden spike in small transfers (graph degree jumped).”

By hackathon end, the MVP should process streaming data and output alerts with evidence. Anything beyond (like full dashboard or legal filing integration) is optional.

Required datasets and synthetic generation

Real Data: No real BOI or bank data will be provided (unspecified by hackathon). Given privacy, teams must work with synthetic or open datasets.

Public/Synthetic Datasets: Options include: SynthAML (16M transactions) ¹, Kaggle's SAML-D (9.5M transactions), PaySim, and AMLSim. These provide realistic transaction flows but may lack explicit mule labels.

Synthetic Data Generation: We recommend using or adapting **SynthAML** or writing a simple simulator:

- **SDV/Tabular GANs:** Use the SDV framework to generate transactions mimicking known distributions. Tools like *Synthesizer* (open-source) can help.
- **Custom Simulator:** Write a Python script to generate accounts and transactions. For mule patterns, designate “mule accounts” and simulate them receiving funds from one source and dispersing to many accounts rapidly.
- **Data Characteristics:** Include multiple channels (IMPS, NEFT, UPI), and inject alerts (e.g. flagged by a rule, maybe an isolated static “KYC alert”).

Labeling: In synthetic data, label accounts flagged as mules. Also create normal account activity. This labeled dataset will train/test the ML component.

Table: Example Data Approaches

Source	Type	Size	Access / Tools	Notes
SynthAML ¹	Synthetic (16M txns, 20k alerts)	Public (Nature)	Download (CSV)	Good base; may need custom mule injection.
PaySim/AMLSim	Simulated transactions	~150k txns	Code on GitHub	Agent-based, can script mule patterns.
Kaggle SAML-D	Synthetic (9.5M txns)	Kaggle platform	Kaggle login	Large, but general AML (no explicit mules).
Custom Generator	User-defined (small)	Team-defined	Python (pandas, faker)	Full control; generate specific mule cases.
BOI/IITH Sample?	Possibly hackathon-provided	Unknown	Provided by organizers? (unspecified)	If any official sample is given, use it.

Table: Datasets and synthetic generation options. Real banking data is not available; all data must be synthetically produced or openly sourced ¹.

Technical approach

We propose a **streaming analytics pipeline**: ingest transactions and alerts in real time, compute features, score accounts, and output alerts. Key components:

- **Streaming ingestion**: Use **Apache Kafka** to simulate or receive transaction streams. Transactions (from all channels) and external alerts (fraud alerts, law enforcement tips) come into separate topics.
- **Feature Engine**: A stream processor (e.g. **Apache Flink** or Kafka Streams) groups transactions by account, computing features like:
 - *Velocity*: number of transactions and total amount in sliding windows (e.g. 1 hour, 24 hours).
 - *Graph features*: Build a mini graph (in-memory or via Neo4j) of account relationships. Compute metrics: out-degree, betweenness centrality, PageRank (from example: PageRank dips for mules ⁴).
 - *Behavioral features*: New payee ratio, KYC-age of account, device/fingerprints (if simulated).
- **Machine Learning Model**: Input these features into a model. Options:
 - **Graph Neural Network (GNN)**: Treat accounts as nodes and transactions as edges. A GNN can learn to classify nodes as mules (as in ⁵).
 - **Sequence Model (LSTM)**: Model an account's transaction time series to detect unusual bursts.
 - **Random Forest / XGBoost**: Train on engineered features for a baseline approach. This may be simpler and explainable.
 - Possibly an ensemble of the above.
- **Explainability**: Use model-agnostic explainers (e.g. SHAP) to highlight which features drove the score. Logging these in alerts will satisfy auditability (e.g. "Alert: account A flagged – due to sudden spike in transfers and high graph centrality change.").
- **Risk Scoring & Alerting**: The model outputs a risk score (0–1). If above a threshold, create an alert record (with account ID, score, key contributing factors) and send to an *Analyst Dashboard*. Teams can implement a simple web UI or CLI printout as the demo.

Mermaid System Architecture

```
graph LR
    subgraph Streams
        T[Transaction Stream]
        R[Rule Alerts Stream]
        F[Fake Regulatory Tips]
    end
    T --> FE
    R --> FE
    F --> FE
    FE[Feature Extraction Engine] --> ML[ML/GNN Models]
    FE --> Rules[Rules Engine (e.g. KYC checks)]
```

```

Rules --> ML
ML --> Score[Risk Score & Explainability]
Score --> AD[Alert Dashboard / Email]

```

Diagram: Simplified architecture. Real-time transaction and alert streams feed a feature engine. This, together with a rule-based engine (applying thresholds, KYC flags), feeds an ML model (could be a GNN or LSTM). The model outputs a risk score and top contributing factors. Alerts are then sent to analysts.

Toolchain & Infrastructure

- **Streaming/Data Ingest:** Apache Kafka (open-source) to queue transactions. Kafka is free and well-documented, fitting a student team's use case.
- **Stream Processing:** Apache Flink or Kafka Streams (Java/Scala) for feature computation. Python teams can use Faust or Spark Structured Streaming, though Java/Scala options might integrate better with Kafka.
- **Graph Database:** Neo4j Community Edition (free) or TigerGraph (free tier) to store account graph if needed. Alternatively, use in-memory graph libraries (NetworkX) for small-scale demo. Neo4j is preferable for graph queries.
- **Database:** A simple relational DB (PostgreSQL) for user profiles. For prototype, even CSV files suffice.
- **ML Frameworks:** Python environment with Pandas/NumPy. For models: Scikit-Learn/XGBoost (fast to train) and PyTorch (for any neural nets). PyTorch Geometric (PyG) can implement GNNs if team is adventurous.
- **Development Tools:** Jupyter/PyCharm for development; GitHub for code repo. Docker for consistent environment (optional but recommended).
- **Visualization:** ELK stack or Grafana for a quick dashboard, or even a simple Flask app to display alerts. Grafana Loki can display logs from the pipeline.
- **Jupyter/Colab:** For data exploration and prototyping.

Table of possible tools:

Category	Tool	Purpose	Maturity
Streaming	Apache Kafka	Ingest transactions/alerts (open-source)	High (widely used)
Stream Processing	Apache Flink/Spark	Real-time feature processing	High
Graph DB	Neo4j Community	Store/query transaction graph	Medium (community)
ML (Tabular)	Scikit-Learn / XGBoost	Quick classifier baselines	High
ML (Neural)	PyTorch / PyTorch Geometric	LSTM/GNN models	High
Dashboard	Grafana / Kibana	Alerts visualization & logs	Medium
DevOps	Docker / Kubernetes	Containerize services	Medium
CIC (RBI data)	<i>None specified</i>	(Likely no direct access; simulate)	Unspecified

Table: Candidate tools. Focus on freely available, widely-used solutions. (Any direct RBI data feeds or enterprise tools are not open to students.)

Integration constraints (BOI/IITH)

- **Data Access:** No official bank data is provided (unspecified). Teams must use simulated data only. If any sample is given by organizers, use it.
- **IP Ownership:** By hackathon rules, any developed code/IP will be co-owned by BOI and IITH ². Teams cannot claim sole ownership.
- **Compliance:** Any solution should respect AML/CFT guidelines. Real implementation would require BOI/FIU approval. Document in your report how an alert meets RBI red-flag criteria.
- **Environment:** Final demo likely runs on IITH infrastructure. Ensure software runs in standard environments (Linux) with open-source stacks.
- **Security:** Do not use any real personal data. Follow IITH/BOI's code-of-conduct and data privacy norms.
- **IP Issues:** Use only open-source libraries; avoid proprietary code.
- **Communication:** For clarifications, contact boihackathon@cse.iith.ac.in ⁶ (as listed on hackathon page).

Any internal BOI constraints (e.g. allowed APIs for core-banking) are unspecified. Assume a “mock” integration: e.g. fake transaction API endpoints.

Practical tips for hackathon

- **Demo strategy:** Prepare clear, realistic scenarios. For example, show account A receiving money then rapidly sending to many small accounts – and demonstrate your system flagging it. Use distinct data in slides vs code to avoid rote.
- **Visualization:** Simple dashboards (Grafana/Kibana) help judges see alert workflow. Even a color-coded log or printed report can suffice.
- **Focus on core functionality:** Judges care more about a working detection prototype than flashy UI. Ensure the pipeline runs end-to-end.
- **Avoid over-engineering:** With limited time, favor simpler models (e.g. random forest + rule) unless advanced models are well-tested.
- **Plagiarism:** All code must be original. No copying from competitors or the web.
- **Teamwork:** Assign clear tasks and integrate frequently. Daily stand-ups in initial weeks help.
- **Documentation:** Keep code well-commented; this eases the progress report and final write-up.
- **Test thoroughly:** Check edge cases (e.g. very high-value legit transactions) to minimize false positives in demo.
- **Failure modes:** Be ready to explain any missed case; know the limitations (data-driven vs rule gaps).

Schedule and milestones

Below is a suggested ~12-week schedule, aligned with hackathon phases ⁷. It assumes the hackathon runs from early May to late August 2026:

Week	Dates	Milestones / Activities
1-2	7–21 May	Team formation, role assignment, tool setup. Study hackathon docs. Draft proposal.

Week	Dates	Milestones / Activities
3	22–31 May	Data preparation: gather synthetic datasets, write generator scripts. Define features (velocity, graph).
4	1–7 June	Build streaming pipeline: Kafka topics, Flink jobs to ingest data into DB/graph.
5	8–14 June	Implement baseline detection: simple rule (e.g. high transfer count) or RF model on static data. Test end-to-end.
6	15–21 June	Improve ML model: add time-series (LSTM) or graph model (if feasible). Continue integration tests.
7	22–30 June	Develop risk scoring logic, add explainability output. Prepare initial alert interface (logs or simple UI).
8	1–7 July	Midpoint review: refine features (e.g. PageRank calcs). Start work on progress report content.
9	8–14 July	Optimize performance (batch vs streaming). Incorporate any feedback. Write half of final presentation.
10	15–21 July	Finalize models. Stress-test with larger simulated streams. Tune thresholds to balance FP/FN.
11	22–31 July	Polish demo: rehearsals, screenshot system in action. Finalize progress report and slides.
12	1–10 Aug	Prepare for finale: create walkthrough script, ensure environment stability. Embed any visuals in slides.
<i>Report Deadline</i>	17 Aug	Submit progress report to hackathon organizers.
13	11–26 Aug	Dry-run final demo. Address last-minute issues.
<i>Final Event</i>	27–28 Aug	Present and demo solution at IITH.

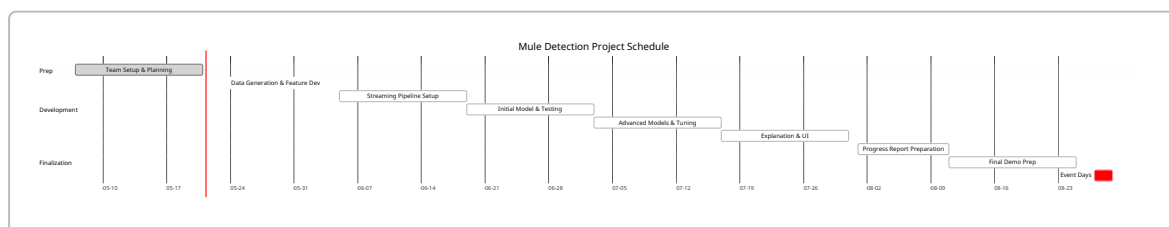


Diagram: Gantt chart of planned activities. Each phase builds on the previous, with final demo in late August.

Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
<i>Data Unavailability</i>	High	High	Use synthetic data (SynthAML, custom sim). Confirm any example data from organizers early.
<i>Steep Learning Curve (AML Domain)</i>	Medium	Medium	Quickly learn basics from RBI/Gov. AML guidelines. Assign one member domain lead.
<i>Model Underperformance</i>	Medium	High	Keep a simple rule-based backup. Hyperparameter tuning; gather more synthetic cases.
<i>Integration Delays</i>	Medium	Medium	Start pipeline early (Kafka/DB). Use simple Docker setups.
<i>Time Management</i>	High	High	Follow the schedule strictly. Prioritize MVP tasks.
<i>Team Coordination Issues</i>	Low/Med	Medium	Daily/weekly stand-ups. Use version control.
<i>Hackathon Rules Misinterpretation</i>	Low	Low	Clarify doubts with organizers (boihackathon@cse.iith.ac.in ⁶) early.

Table: Main project risks and mitigation strategies. Synthetic data usage and early pipeline setup are critical to avoid delays.

Assumptions

- **No official data/API:** It is assumed BOI will not share real transaction data. All data used is synthetic or public.
- **Mentorship Unspecified:** No formal mentorship schedule is given, so assume self-learning and online support.
- **Hackathon Scope:** Focus is solely on the given problem; integration with BOI's production systems is *not* required.
- **Team Skills:** Team members have basic programming and database knowledge; any advanced ML or domain gaps must be filled by self-study.
- **No Cost Constraints:** Use of open-source/free tools is assumed sufficient; no budgeting is needed.

Sources: Hackathon problem statement ³ and rules (IP shared ²) informed scope. RBI/AML regulations (RBI circular Dec 2010 on mules) provide context ⁸. SynthAML provides reference for synthetic AML data ¹. Contact boihackathon@cse.iith.ac.in ⁶ for any clarifications.

¹ A synthetic data set to benchmark anti-money laundering methods | Scientific Data
[https://www.nature.com/articles/s41597-023-02569-2?](https://www.nature.com/articles/s41597-023-02569-2?error=cookies_not_supported&code=b2406a12-49bd-45d9-8924-4bae51c9a386)

[https://www.nature.com/articles/s41597-023-02569-2?](https://www.nature.com/articles/s41597-023-02569-2?error=cookies_not_supported&code=b2406a12-49bd-45d9-8924-4bae51c9a386)

² ³ ⁶ ⁷ CyberShield LIVE Instance

<https://boihackathon.cse.iith.ac.in/hackathon2026/hackathonReg/newReg/>

4 **Mule Account Mitigation - Neo4j Industry Use Cases**

<https://neo4j.com/developer/industry-use-cases/whitepapers/finserv/fraud/mule-account-mitigation/>

5 **Detection of Mule Accounts and Fraudsters in UPI Transactions Using AI and Machine Learning Techniques - IRE Journals**

<https://www.irejournals.com/paper-details/1716166>

8 **Money Mules: FinCrime's Trojan Horse Unveiled**

<https://assets.kpmg.com/content/dam/kpmgsites/in/pdf/2024/07/money-mules-fincrimet-trojan-horse-unveiled.pdf.coredownload.inline.pdf>